

ARTISANS

On-Demand Artisan Platform — Ghana

Project Completion Checklist

Flutter · Express.js · Supabase

Kwabena Boateng Konadu	3395422	Express.js Backend
Nhyira Esi Bodwewa Andoh	3370722	Flutter Frontend
Peniel Osei Okrah	3406022	Supabase Layer

Supervisor: Prof. Gaddafi Abdul-Salaam — KNUST Computer Science

How to Use This Document

This checklist covers every task required to deliver Artisans from environment setup to a validated MVP and a submitted final year project. Tasks are grouped by phase and owner but are not strictly ordered within each group — work through them in the sequence that makes sense for your current state.

Owner abbreviations:

Kwabena	Express.js backend — business logic, matching, notifications
Nhyira	Flutter frontend — all screens, maps, state management
Peniel	Supabase layer — schema, RLS, realtime, storage
All	All three members involved or aware
Team	Group decision or collaborative effort required

The Future section at the end lists items deliberately removed from the project scope — WhatsApp dispatch, USSD, payments, and post-launch features. Do not work on these until the MVP is validated and the project is submitted.

PHASE 0 — Environment & Accounts Setup

All three members. Before writing any code.

	Task	Owner
☐	Create Supabase project at supabase.com — save URL, anon key, and service_role key securely	Peniel
☐	Create Firebase project — enable Cloud Messaging (FCM) for push notifications	Kwabena
☐	Create Google Cloud project — enable Maps SDK for Android, Maps SDK for iOS, Directions API, Places API	Nhyira
☐	Create shared GitHub repository — three folders: /api /app /supabase	Team
☐	Set up branch strategy: feature/* → dev → main, PR required for all merges	Team
☐	Store all API keys in a shared password manager — never commit to repo	Team
☐	Install Flutter SDK (stable channel) — run flutter doctor, resolve all issues	Nhyira
☐	Install Android Studio + Android SDK + create Android emulator (Pixel 6, API 33)	Nhyira
☐	Install VS Code with Flutter, Dart, and Riverpod Snippets extensions	Nhyira
☐	Install Node.js 20 LTS — verify with node -v and npm -v	Kwabena
☐	Install TypeScript globally: npm install -g typescript ts-node nodemon	Kwabena
☐	Install Supabase CLI: npm install -g supabase	Peniel

PHASE 1A — Supabase Layer (Peniel)

Must be complete before Kwabena or Nhyira can integrate.

	Task	Owner
☐	Create migration file: supabase migration new initial_schema	Peniel
☐	Write SQL — profiles table: id (FK auth.users), full_name, phone, avatar_url, role, fcm_token	Peniel
☐	Write SQL — workers table: skills[], hourly_rate, rate_type, is_available, is_verified, current_lat, current_lng, location_at, rating, total_jobs, service_areas[]	Peniel
☐	Write SQL — categories table: name, slug, icon_name, sort_order, is_active	Peniel
☐	Write SQL — jobs table: client_id, worker_id, category_id, title, description, photo_urls[], location_lat, location_lng, address_label, status, job_mode, budget_type, budget_fixed, budget_min, budget_max, scheduled_for, expires_at, service_type	Peniel
☐	Write SQL — job_applications table: job_id, worker_id, message, proposed_rate, status (for Post-a-Job mode)	Peniel
☐	Write SQL — reviews table: job_id (UNIQUE), reviewer_id, worker_id, rating (1–5), comment	Peniel
☐	Write SQL — messages table: job_id, sender_id, content, is_read (for in-app chat, Phase 2)	Peniel
☐	Run: supabase db push — confirm all tables visible in Supabase dashboard	Peniel
☐	Enable Row Level Security on every table — no table left unprotected	Peniel
☐	RLS — profiles: user reads and updates own row only	Peniel
☐	RLS — workers: public SELECT for discovery feed, own UPDATE only	Peniel
☐	RLS — jobs: client sees own jobs, worker sees matched jobs, Express uses service_role for all writes	Peniel
☐	RLS — messages: only client_id and worker_id of that job can read or write	Peniel
☐	RLS — reviews: only client of a completed job can INSERT	Peniel
☐	Test RLS: with a test user JWT, confirm you cannot read another user's jobs	Peniel
☐	Write trigger: update_worker_rating() — recalculates rating average after INSERT on reviews	Peniel
☐	Write trigger: set_job_expiry() — sets expires_at = now() + 15 minutes for ASAP jobs on INSERT	Peniel
☐	Write trigger: increment_total_jobs() — increments workers.total_jobs when job status → completed	Peniel
☐	Enable Realtime on jobs table: ALTER TABLE jobs REPLICA IDENTITY FULL	Peniel
☐	Enable Realtime on workers table: ALTER TABLE workers REPLICA IDENTITY FULL	Peniel
☐	Enable Realtime on messages table: ALTER TABLE messages REPLICA IDENTITY FULL	Peniel
☐	Test Realtime: update a row manually in dashboard, confirm client receives postgres_changes event	Peniel
☐	Create storage bucket: avatars — public read, max 2MB per file	Peniel
☐	Create storage bucket: job-photos — authenticated read only, max 5MB per file	Peniel

☐	Create storage bucket: id-docs — admin read only, max 10MB per file	Peniel
☐	Generate TypeScript types: supabase gen types typescript > supabase/types.ts — commit to repo	Peniel
☐	Seed: insert 5 service categories (Home Services, Technical & Repairs, Personal, Transport, Beauty)	Peniel
☐	Seed: insert 3 test worker accounts with skills, location coords (Kumasi), is_available = true	Peniel
☐	Seed: insert 2 test client accounts	Peniel

PHASE 1B — Express.js Backend (Kwabena)

Depends on Peniel completing Phase 1A schema first.

	Task	Owner
☐	Scaffold project: <code>npx express-generator --no-view artisans-api</code> , convert to TypeScript	Kwabena
☐	Install dependencies: <code>express</code> , <code>@supabase/supabase-js</code> , <code>firebase-admin</code> , <code>node-cron</code> , <code>axios</code> , <code>zod</code> , <code>cors</code> , <code>helmet</code> , <code>express-rate-limit</code>	Kwabena
☐	Create <code>config/supabase.ts</code> — <code>service_role</code> client instance (bypasses RLS for server operations)	Kwabena
☐	Create <code>config/firebase.ts</code> — initialise <code>firebase-admin</code> with service account JSON	Kwabena
☐	Configure <code>.env</code> : <code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code> , <code>FIREBASE_SERVICE_ACCOUNT_PATH</code> , <code>PORT</code>	Kwabena
☐	Write <code>middleware/auth.ts</code> — verify Supabase JWT on every protected route	Kwabena
☐	Write <code>middleware/errorHandler.ts</code> — global handler, always returns structured JSON error	Kwabena
☐	Write <code>middleware/rateLimiter.ts</code> — 100 requests per minute per IP	Kwabena
☐	Apply <code>cors()</code> , <code>helmet()</code> , <code>rateLimiter</code> to all routes in <code>index.ts</code>	Kwabena
☐	POST <code>/api/jobs/create</code> — validate body with <code>zod</code> , insert job via <code>service_role</code> , trigger matching for ASAP jobs	Kwabena
☐	POST <code>/api/jobs/:id/cancel</code> — update status to cancelled, send FCM to matched worker if one exists	Kwabena
☐	POST <code>/api/jobs/:id/complete</code> — update status to completed, send FCM rating prompt to client	Kwabena
☐	PUT <code>/api/workers/location</code> — update <code>current_lat</code> , <code>current_lng</code> , <code>location_at</code> in workers table	Kwabena
☐	PUT <code>/api/workers/availability</code> — toggle <code>is_available</code> , clear location timestamp if going offline	Kwabena
☐	POST <code>/api/workers/accept/:jobId</code> — atomic status update to matched, returns 409 if already taken	Kwabena
☐	POST <code>/api/workers/decline/:jobId</code> — log decline, trigger re-dispatch to next set of workers	Kwabena
☐	GET <code>/api/workers/nearby</code> — query available workers within radius, return ranked by distance	Kwabena
☐	Write <code>services/matchingService.ts</code> — Haversine distance formula function	Kwabena
☐	Implement <code>findAndDispatch(jobId, round=1)</code> — query workers by category, <code>is_available</code> , <code>is_verified</code> , proximity	Kwabena
☐	Implement ranking: sort by distance ASC then rating DESC, take top 3	Kwabena
☐	Implement 3-rounds-of-3 dispatch — broadcast to top 3 simultaneously, wait 90 seconds, retry next 3	Kwabena
☐	Implement <code>expireJob(jobId)</code> — after 3 rounds with no acceptance, set status to expired	Kwabena
☐	Implement <code>checkAndReDispatch()</code> — called by <code>setTimeout</code> after 90s, checks if job still searching	Kwabena
☐	Write <code>services/notifyService.ts</code> — send FCM push to a single device token via <code>firebase-admin</code>	Kwabena
☐	<code>notifyWorkerNewJob(worker, job)</code> — FCM push with job title, location, <code>jobId</code> in data payload	Kwabena

☐	notifyJobMatched(clientFcmToken, workerName) — FCM push to client when worker accepts	Kwabena
☐	notifyJobExpired(clientFcmToken) — FCM push to client after all dispatch rounds exhausted	Kwabena
☐	notifyJobCompleted(clientFcmToken) — FCM push prompting client to leave a review	Kwabena
☐	notifyScheduledReminder(clientFcmToken, workerName) — FCM push 24 hours before scheduled job	Kwabena
☐	Test FCM delivery end-to-end to a real Android device	Kwabena
☐	Write services/schedulerService.ts — node-cron job every minute: expire ASAP jobs past expires_at	Kwabena
☐	Write node-cron job every hour: send FCM reminders for scheduled jobs in next 24 hours	Kwabena
☐	Write Postman collection for all endpoints — share with team in /api/postman/ folder	Kwabena
☐	Test matching flow end-to-end: create ASAP job → matching fires → FCM received on test device	Kwabena
☐	Test atomic accept: simulate two workers accepting the same job simultaneously — confirm only one succeeds	Kwabena

PHASE 1C — Flutter Frontend (Nhyira)

Auth screens can start immediately. Rest depends on Phase 1A and 1B.

	Task	Owner
☐	Create Flutter project: flutter create artisans_app --org com.artisans	Nhyira
☐	Set up feature-first folder structure: features/auth, home, job_post, booking, worker, profile, chat	Nhyira
☐	Add all dependencies to pubspec.yaml: supabase_flutter, go_router, flutter_riverpod, riverpod_annotation, google_maps_flutter, geolocator, geocoding, dio, firebase_messaging, flutter_local_notifications, image_picker, cached_network_image, permission_handler, connectivity_plus, intl, shared_preferences	Nhyira
☐	Add google-services.json (Android) and GoogleService-Info.plist (iOS) from Firebase console	Nhyira
☐	Add Google Maps API key to AndroidManifest.xml and AppDelegate.swift/Info.plist	Nhyira
☐	Initialise Supabase in main.dart with project URL and anon key	Nhyira
☐	Write core/theme/app_colors.dart — dark background (#0F0F0F), surface (#1A1A2E), primary purple (#7C4DFF), accent blue (#2979FF), success green, text colors	Nhyira
☐	Write core/theme/app_text_styles.dart — heading, body, caption, rate text styles	Nhyira
☐	Write core/theme/app_theme.dart — dark ThemeData, apply to MaterialApp	Nhyira
☐	Write shared/widgets/liquid_card.dart — animated press scale, rounded corners, dark surface color	Nhyira
☐	Write shared/widgets/gradient_button.dart — purple to blue LinearGradient CTA button	Nhyira
☐	Write shared/widgets/step_progress_bar.dart — for 7-step job post flow	Nhyira
☐	Write shared/widgets/worker_card.dart — name, skill, star rating, €/hr rate badge, Available/Unavailable status badge	Nhyira
☐	Write shared/widgets/category_chip.dart — icon + label, selectable, accent colour per category	Nhyira
☐	Write core/router/app_router.dart — all routes declared in GoRouter	Nhyira
☐	Implement auth redirect in go_router: unauthenticated users → /auth/login	Nhyira
☐	Implement role guard: worker-side routes blocked for client accounts and vice versa	Nhyira
☐	Build auth/screens/onboarding_screen.dart — two LiquidCards: "I offer services" and "I need a worker"	Nhyira
☐	Build auth/screens/login_screen.dart — phone entry with +233 Ghana prefix, gradient button	Nhyira
☐	Build auth/screens/otp_screen.dart — 6 separate input boxes, auto-advance between boxes, resend countdown	Nhyira
☐	Connect OTP screens to Supabase signInWithOtp() and verifyOtp()	Nhyira
☐	Test full auth cycle on a real Android device	Nhyira
☐	Build home/screens/home_screen.dart — greeting with user name, search bar, two gradient action buttons (Find a Worker / Post a Job), horizontal category chips, worker card feed	Nhyira

☐	Write home/providers/workers_provider.dart — Riverpod FutureProvider calling GET /api/workers/nearby	Nhyira
☐	Handle empty state: no workers available near you — show illustration, not blank screen	Nhyira
☐	Build home/screens/map_discovery_screen.dart — full-screen Google Map	Nhyira
☐	Render available workers as custom circular photo markers using BitmapDescriptor.fromBytes	Nhyira
☐	Tap marker → show worker mini-card at bottom using DraggableScrollableSheet	Nhyira
☐	Animate camera to user current GPS position on screen open	Nhyira
☐	Build home/screens/worker_profile_screen.dart — full profile: photo, name, verified badge, skills, rating stars, \$/hr rate, bio, review list, Book Now button	Nhyira
☐	Build job_post/screens/job_post_flow.dart — PageController shell, StepProgressBar, 7 children	Nhyira
☐	Step 1 — CategoryStep: scrollable grid of service category chips	Nhyira
☐	Step 2 — SubcategoryStep: skill chips for selected category (e.g. Electrician, Plumber under Technical)	Nhyira
☐	Step 3 — TitleStep: single text field, 80 character limit with counter	Nhyira
☐	Step 4 — DescriptionStep: multiline text field + image_picker for up to 3 photos	Nhyira
☐	Step 5 — LocationStep: Google Map with draggable pin, reverse geocoding fills address label below	Nhyira
☐	Step 6 — BudgetScheduleStep: SegmentedControl for Fixed/Range/Negotiable budget + ASAP/Scheduled/Flexible urgency + DateTimePicker when Scheduled selected	Nhyira
☐	Step 7 — ReviewSubmitStep: full job summary card, contact preference selector, Post Job button	Nhyira
☐	Write job_post/providers/job_post_provider.dart — StateNotifier accumulating state across all 7 steps	Nhyira
☐	On submit: POST to Express /api/jobs/create, on success navigate to /job/searching with jobId	Nhyira
☐	Build booking/screens/searching_screen.dart — pulsing ring animation, "Finding your artisan" text, cancel button	Nhyira
☐	Subscribe to Supabase Realtime channel job:{jobId} — listen for status changes	Nhyira
☐	On status → matched: auto-navigate to /job/tracking	Nhyira
☐	On status → expired: show "No workers found" state with retry and cancel options	Nhyira
☐	Build booking/screens/tracking_screen.dart — Google Map with worker marker and client pin	Nhyira
☐	Write booking/providers/tracking_provider.dart — Riverpod StreamProvider wrapping Supabase Realtime on workers table filtered by worker_id	Nhyira
☐	Animate worker marker smoothly between incoming LatLng updates	Nhyira
☐	Call Google Directions API once on screen load — draw route polyline between worker and client pin	Nhyira
☐	Display ETA card at bottom: worker photo, name, estimated time, call button	Nhyira
☐	Call button: launch dialler with worker phone number via url_launcher	Nhyira
☐	Build booking/screens/review_screen.dart — tap star rating, optional comment field, submit button	Nhyira

☐	On submit: INSERT to Supabase reviews table, then POST /api/jobs/:id/complete to Express	Nhyira
☐	Build worker/screens/worker_home_screen.dart — availability toggle at top, incoming job request cards	Nhyira
☐	Subscribe to Supabase Realtime — incoming jobs matching worker category and proximity	Nhyira
☐	Build worker/screens/worker_job_detail_screen.dart — job title, description, photos, client location on map, budget, Accept and Decline buttons	Nhyira
☐	Accept: POST /api/workers/accept/:jobId — navigate to active job screen on 200 response	Nhyira
☐	Decline: POST /api/workers/decline/:jobId	Nhyira
☐	Build worker/screens/worker_active_job_screen.dart — map with client pin, Mark as Started button, Mark as Complete button	Nhyira
☐	Worker location ping: Timer.periodic every 10 seconds calling PUT /api/workers/location while is_available = true	Nhyira
☐	Build profile/screens/profile_screen.dart — avatar upload, name, phone, verification status tabs	Nhyira
☐	Build profile/screens/booking_history_screen.dart — past jobs list, clean empty state for new users	Nhyira
☐	Write core/api/dio_client.dart — base URL config, JWT interceptor attaching Supabase access token to every request	Nhyira
☐	Implement retry interceptor in Dio: 3 attempts, exponential backoff (500ms, 1s, 2s)	Nhyira
☐	Implement offline banner: connectivity_plus detects no network → show persistent top banner	Nhyira

PHASE 2 — Integration & End-to-End Testing

All three members working together. Connect the three layers.

	Task	Owner
☐	Confirm Flutter authenticates with Supabase and receives a valid JWT	All
☐	Confirm Flutter can call Express endpoints using the Supabase JWT — get 200 response	All
☐	Confirm Express writes to Supabase using service_role key without RLS blocking	All
☐	Confirm Supabase Realtime event reaches Flutter when Express updates a job status row	All
☐	Full ASAP flow: client posts job → Express matches → worker FCM fires → worker accepts → client tracking screen shows worker location	All
☐	Full Scheduled flow: client schedules job → Express cron sends FCM reminder 24hrs before → flow proceeds normally	All
☐	Test double-accept: two worker devices attempt to accept the same job — only first succeeds, second gets 409	All
☐	Test job expiry: no worker accepts across 3 rounds — client receives expired FCM notification	All
☐	Test GPS denied: manual address input renders and creates job correctly	All
☐	Test network drop mid-request: Dio retry logic recovers, job not duplicated in DB	All
☐	Test on low-spec Android device (Tecno or Infinix class) — no UI jank or crashes	Nhyira
☐	Test worker location updates visible on client tracking map within 15 seconds on 3G connection	All
☐	Test photo upload: client uploads job photo, URL stored in jobs.photo_urls in DB	Nhyira
☐	Test RLS: attempt to read another user's jobs from Flutter client — confirm 0 rows returned	Peniel
☐	Test FCM delivery reliability on MTN Ghana and Vodafone Ghana networks	All
☐	Test category filter: client in Home category only sees workers with home skill set	All

PHASE 3 — Error States, Polish & Edge Cases

Every screen needs a loading state, empty state, and error state.

	Task	Owner
☐	Every data-fetching screen shows a skeleton loader — never a blank white screen	Nhyira
☐	Job post submit button shows loading spinner while POST is in flight	Nhyira
☐	Worker accept button disabled immediately after first tap — prevent duplicate POST	Nhyira
☐	Network error on job create: snackbar "Failed to post — tap to retry"	Nhyira
☐	Session expired mid-use: Supabase auth error → auto-redirect to login without crash	Nhyira
☐	FCM delivery failure: log event to Supabase for debugging — do not crash Express	Kwabena
☐	All Express errors return structured JSON { error: string, code: string } — never raw stack trace to client	Kwabena
☐	Offline banner appears within 2 seconds of network loss	Nhyira
☐	Worker location ping fails silently on network loss — does not crash app or show error	Nhyira
☐	Worker arrives — client can still call worker directly via phone button if map lags	Nhyira
☐	All money values display as ₵30 not 30.0	Nhyira
☐	All ratings display as 4.8 not 4.800000	Nhyira
☐	All timestamps display as "2 hours ago" not ISO string	Nhyira
☐	All images use CachedNetworkImage with a shimmer placeholder while loading	Nhyira
☐	Haptic feedback on: role selection confirm, job submit, worker accept, worker decline	Nhyira
☐	Worker availability toggle shows immediate optimistic UI update before API confirms	Nhyira
☐	Searching screen cancel: job status updated to cancelled in DB, worker notified via FCM	All

PHASE 4 — Academic Deliverables

For supervisor review and final year examination submission.

	Task	Owner
☐	Write Software Requirements Specification (SRS) — functional and non-functional requirements	Team
☐	Write Feasibility Study — technical, economic, operational, schedule, legal analysis	Team
☐	Write System Design chapter — architecture diagram, data flow diagram, database ER diagram	Team
☐	Write Implementation chapter — code structure, library justifications, key technical decisions	Team
☐	Write Testing chapter — test plan, test cases per requirement, pass/fail results table	Peniel
☐	Write Evaluation chapter — map each requirement to evidence of it being met	Team
☐	Create ER diagram of all Supabase tables with relationships and cardinality	Peniel
☐	Create system architecture diagram: Flutter ↔ Express ↔ Supabase with data flow arrows	Team
☐	Create screen flow diagram connecting all screens with navigation paths	Nhyira
☐	Record 5-minute demo video: full ASAP client flow from home screen to tracking screen	Team
☐	Record 2-minute demo video: worker side — receives FCM, accepts, navigates to job site	Team
☐	Prepare a demo APK distributable via QR code for presentation day	Nhyira
☐	Interim Demo 1 (Supervisor): auth flow working, database schema live, project structure explained	All
☐	Interim Demo 2 (Supervisor): full client request flow demonstrated on physical device	All
☐	MVP Demo (Supervisor): end-to-end flow with real users — job posted and dispatched live	All
☐	Final project report — all chapters complete, reviewed, formatted to department standard	Team

PHASE 5 — Pre-Launch Validation

Real people. Real phones. Kumasi. Before any public release.

	Task	Owner
☐	Talk to 5 handypeople in Kumasi in person — confirm willingness to use the worker app	Team
☐	Talk to 10 potential clients — confirm they would use the app for urgent home fixes	Team
☐	Confirm 3 workers: 1 plumber, 1 electrician, 1 general — names and phone numbers obtained	Team
☐	Confirm 5 beta clients by name who have agreed to test the app	Team
☐	Build preview APK: <code>eas build --platform android --profile preview</code>	Nhyira
☐	Install APK on 3 confirmed worker devices — confirm FCM notifications received	All
☐	Run 5 live end-to-end test requests with real users — document each session	All
☐	After each session: collect verbal feedback, log what broke, fix before next session	All
☐	Confirm GPS accuracy is acceptable in Adum, Bantama, and Suame areas of Kumasi	All
☐	Document the 3 most reported issues — fix all before final project presentation	Team

FUTURE — After Project Completion

Not in scope for the final year project. Revisit after validation.

	Task	Owner
☐	WhatsApp job dispatch for workers without smartphones	Kwabena
☐	USSD self-registration shortcode (*384*ARTISANS#) via MTN Ghana	Kwabena
☐	Mobile Money (MoMo) payment integration via Paystack Ghana	Team
☐	Worker subscription model — monthly fee for priority matching	Team
☐	In-app chat between client and worker (messages table already exists in DB)	Nhyira
☐	Roadside assistance vertical — vehicle details, mechanic and towing categories	Team
☐	Admin web dashboard (Next.js) replacing manual Supabase dashboard usage	Team
☐	Multi-city expansion to Accra	Team
☐	Worker earnings dashboard — total earned, job history, pending ratings	Nhyira
☐	Community radio campaign and trade association partnership for worker recruitment	Team